Journal of Universal Computer Science, vol. 32, no. 8 (2026), 1150-1172
submitted: 21/10/2025, accepted: 16/1/2026, appeared: 28/8/2026 CC BY 4.0

A Proactive Fault Tolerance Approach for Saga Transactions in Microservices Using a Health Prediction Model

Jiangke Wu
(Northern Arizona University, Flagstaff, United States of America
https://orcid.org/0009-0001-5044-6786, jw2966@nau.edu)

Xiaojun Chen
(Zhejiang University, Hangzhou, China
https://orcid.org/0009-0004-7596-8881, chenxiaojun@zju.edu.cn)

Rongxing Shi
(Nanjing Institute of Technology, Nanjing, China
https://orcid.org/0009-0008-4641-5005, srx@njit.edu.cn)

Abstract: The Saga pattern guarantees eventual consistency in microservices but depends on passive post-failure compensation, resulting in resource inefficiency and possible data discrepancies. This study introduces the ”Proactive Saga Failure Avoidance (PSFA)” system, which incorporates deep learning-based health evaluation into the Saga orchestrator. The platform employs an LSTM model to monitor real-time service metrics, enabling the prediction of imminent failures prior to transaction execution. Upon detection of high risk, the transaction is proactively terminated to prevent erroneous computation and intricate rollbacks. Experimental findings obtained in a controlled environment indicate that PSFA detects failure precursors with an accuracy of 1.00 and a recall of 0.99. This method provides an innovative alternative to improve the reliability and efficiency of distributed transactions in microservice settings.

Keywords: Microservices, Distributed Transactions, Saga Pattern, Proactive Fault Tolerance, LSTM, AIOps, Predictive Maintenance
Categories: H.2, I.2, I.5, L.2, L.3, D.2, D.4
DOI: 10.3897/jucs.175609

1 Introduction

With the rapid advancement of digital transformation, microservice architecture has become the mainstream architecture paradigm for modern complex application development with its core advantages such as high cohesion, low coupling and independent deployment [Newman 2015, Nadareishvili et al. 2016, Richardson 2018]. However, while improving development agility, service decomposition also raises a significant challenge of maintaining data consistency across distributed environments, the traditional ACID transaction model cannot cross multiple independent service boundaries and struggles to support the consistency needs of distributed scenarios. In this context, distributed transaction solutions to ensure final consistency were born, among which the Saga pattern has become one of the most widely used solutions in the industry with its efficient coordination ability for distributed transactions [Richardson 2018, Wu 2024].

The Saga pattern achieves eventual consistency by decomposing a long transaction into independent local transactions, each with a corresponding compensation operation [Garcia-Molina and Salem 1987], if any local transaction in the sequence fails, Saga will trigger the compensation logic of the previous successful transaction in reverse order. Although this model can effectively ensure the consistency of distributed scenarios, its essence is a passive, post-failure fault-tolerant strategy, and the core limitation is that it can only intervene after the failure occurs as shown in Figure 1. This passivity leads to three key defects:

- Significant resource underutilization: If the failure occurs in the later stage of the transaction chain, the computing and storage resources consumed by the successful steps in the early stage will be completely wasted;
- The compensation process is resource-intensive: the compensation operation requires additional resources, and under complex business logic, the compensation itself may fail, further aggravating the system burden;
- Insufficient transaction isolation: Due to the lack of transaction isolation, the intermediate state of local transaction submission is visible to other concurrent processes, resulting in data consistency problems, such as ”dirty reads,” before the Saga is completed or fully compensated.

Although active fault avoidance is the core goal of the active fault tolerance mechanism [Salfner et al. 2010], the existing research mostly focuses on the optimization of the compensation process after the failure [Soldani et al. 2018], and the more fundamental solution space of ”fault prevention from the source” has not been fully explored.

To address fill this research gap, this paper proposes a paradigm shift from ”post-fault compensation” to ”pre-fault avoidance”. We proposed the Proactive Saga Failure Avoidance (PSFA) framework, an innovative mechanism specially designed to improve the reliability and efficiency of distributed transactions. The core of the framework is to empower the Saga orchestrator with predictive intelligence: before triggering a critical or high-cost service call, the orchestrator evaluates the failure probability of the target service in real time by calling the prediction service; if the risk value exceeds the predefined threshold, the orchestrator will actively abort the entire Saga transaction and start the instant compensation process, instead of continuing to call the service that is on the verge of failure. This strategic shift as shown in Figure 2 prevents the system from entering an invalid state that would require complex recovery logic, thereby achieving a ’forward shift’ in fault response.

The realization of the PSFA framework is built with the prediction service based on deep learning as the core, which assumes the function of real-time health monitoring. By analyzing the service performance telemetry sequence, it identifies the characteristic patterns that foreshadow the imminent failure; these health indicators, including CPU utilization, memory consumption, disk I/O and other key system-level indicators, directly constitute the key precursor signals before the failure. The framework adopts the long- and short-term memory (LSTM) network (a model that naturally adapts to timing analysis tasks) to accurately capture complex nonlinear associations in telemetry data and to achieve high-precision prediction of the service’s health trajectory. By deeply embedding this prediction ability into the transaction control flow, the PSFA framework can support fine-grained, real-time and active fault-tolerant decision-making, and push fault avoidance from ”responsive” to ”predicted”.

The main contributions of this paper can be summarized as the following three points:

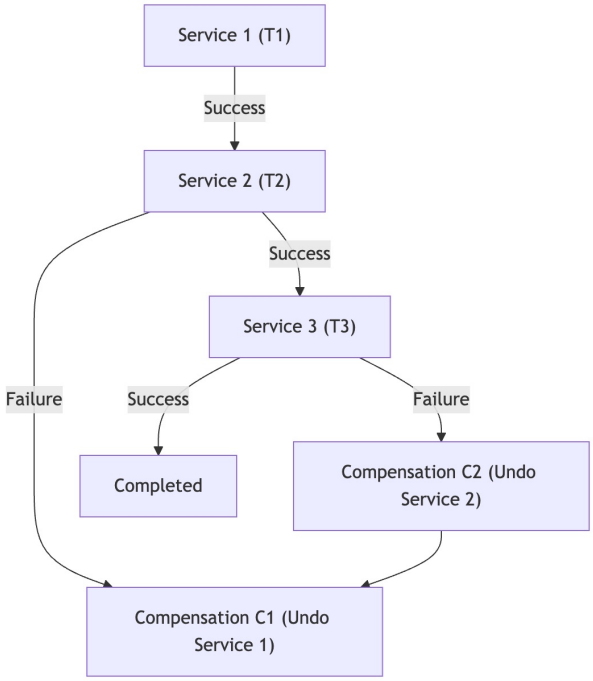


Figure 1: The operation mechanism process of the traditional Saga pattern. This figure shows the specific logic of the system rolling back the submitted transaction changes through passive triggering compensation operations C2 and C1 when the execution of transaction T3 fails, and finally achieves the final consistency guarantee.

- The first predictive fault tolerance framework (PSFA) for Saga pattern, by deeply embedding the predictive analysis capability into the control flow of the Saga pattern, realize the active fault tolerance mechanism, and promote the fundamental transformation of fault tolerance logic from ”passive compensation” to ”active avoidance”.
- Build a high-fidelity fault data generation system, design and implement simulation schemes covering a variety of fault types, accurately simulate the precursor characteristics of the whole process of fault evolution in the real distributed system, and provide an empirical data set with scenario representation for the training of high-reliability prediction models.
- Through systematic experiments to verify the effectiveness of the method, the prediction method based on LSTM obtains quantitative results with high accuracy and high recall rate in the potential service fault identification task, which fully supports the technical feasibility and engineering practical value of the PSFA framework.

The structure of the follow-up content of this paper is arranged as follows: Section 2 systematically reviews the core research progress in the field of Saga pattern and predictive maintenance; Section 3 elaborates in depth the architectural design logic and

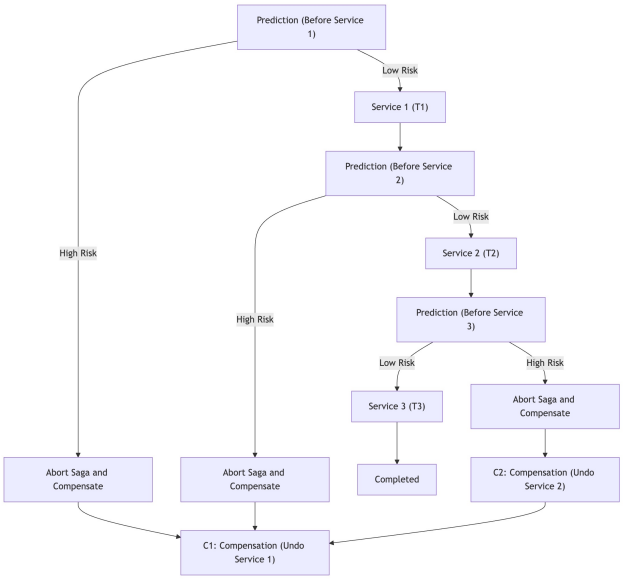


Figure 2: The PSFA framework proposed in this paper actively avoids the operation process. Unlike the passive response mechanism of the traditional model, the process strategically embeds the prediction link before triggering the key service call, by evaluating the probability of failure of the target service in real time, the driving system changes from the passive fault tolerance of "post-fault compensation" to the active defense paradigm of "pre-fault avoidance".

methodology system of the PSFA framework proposed; Section 4 fully presents the experimental setting, quantitative results and multi-dimensional analysis process; Section 5 discusses the theoretical significance and technical limitations of this study; Section 6 summarizes the full text and distills the core conclusions and future prospects.

2 Related Work

2.1 The Saga Pattern: Fundamentals, Challenges, and Recent Advances

The Saga pattern was pioneered by Garcia-Molina and Salem in 1987. The core goal is to solve the challenges of data consistency in long-life cycle transaction scenarios [Garcia-Molina and Salem 1987]. In the contemporary microservice architecture, the engineering implementation of this model mainly relies on two technical paradigms: choreography and orchestration [Richardson 2018]. The choreography paradigm relies on the "release-subscription" event mechanism between services to drive transaction flow, although it achieves structural decentralization, the complexity of transaction chain tracking and debugging has increased exponentially with the expansion of the service scale, representing a significant challenge in distributed transaction management. In stark contrast, the orchestration paradigm naturally has better full-link observability and process controllability through the centralized arrangement to coordinate service

call and decision-making logic. The PSFA framework proposed in this paper is based on the orchestration-based Saga design, the ”central node” attribute of the centralized orchestrator, which provides an ideal architectural integration point for embedding predictive decision-making logic, and supports the implementation of the ”active fault avoidance” function from the mechanism level.

Although the Saga pattern has been widely used, its core limitation is still that it cannot meet the isolation requirements of ACID transactions, because each local transaction will immediately submit changes, its temporary status will be concurrently issued by others before the completion of the entire Saga. Transaction perception directly causes data consistency problems such as ”dirty reading” and ”lost updates”. In addition, when the complexity of the system increases, the difficulty of debugging and tracking of the failed Saga process increases exponentially [Soldani et al. 2018]. In response to these challenges, the academic and industrial circles have put forward a variety of optimization schemes: for example, some studies have temporarily stored transaction operations in memory by introducing quota cache, submission synchronization services and other mechanisms, and only persist in the database after the success of the whole Saga to alleviate the problem of insufficient isolation. However, the common focus of these studies is still on improving the data security of Saga transactions in failure scenarios. In contrast, this study cuts into the new dimension of fault avoidance, reducing the occurrence of faults from the root cause through predictive analysis, so as to avoid the system falling into a complex compensation and data recovery process, and avoids the limitations of the ”post-processing” of the traditional scheme from the mechanism level.

2.2 AIOps and Predictive Maintenance

In the face of dynamically evolved and highly complex distributed systems, the limitations of traditional IT monitoring methods based on static thresholds are increasingly prominent, their preset fixed thresholds are difficult to adapt to the dynamic changes of the system state, resulting in abnormal perception lag, fault location and other problems. Against this technological background, AIOps (Artificial Intelligence-Driven IT Operation and Maintenance) was born as an emerging paradigm. Its core logic lies in the deep integration of machine learning and big data technology, and in-depth mining and pattern recognition of multi-dimensional operation and maintenance data such as indicators, logs and links, so as to support the realization of high-order automated operation and maintenance capabilities such as abnormal detection, root cause analysis and fault prediction. Provide intelligent solutions for the stability of distributed systems. In the field of distributed system fault prediction, early research focuses on traditional machine learning algorithms such as support vector machine (SVM), decision tree, Bayesian network, etc., and predicts failures at the virtual machine or container level [Cortes and Vapnik 1995, Breiman 2001]. In recent years, with the rapid development of deep learning, the use of neural network models for predictive maintenance has become a hot spot for field research. A large number of studies have confirmed that deep learning models can more efficiently extract complex fault patterns from high-dimensional nonlinear monitoring data [Zhang et al. 2018, LeCun et al. 2015, Goodfellow et al. 2016]; the successful practice of many real scenarios has also confirmed the core value of AIOps, it can significantly reduce the average repair time (MTTR) and minimizing service downtime.

Recent advancements in pattern recognition underscore the robustness of deep learning in handling complex, non-linear monitoring data across diverse domains. For instance,

artificial neural networks utilizing backpropagation have demonstrated exceptional accuracy in gesture-based classification by effectively managing noisy and error-prone sensor data [Irvanizam et al. 2023], while integrated architectures using face mesh technologies have significantly enhanced detection capabilities under varying illumination and background conditions [Hangaragi and Singh 2023]. Furthermore, deep learning continues to excel in specialized engineering tasks, such as achieving high-precision object localization through modified VGG architectures [Correia and Jardim 2025] and modeling temporal morphological changes to ensure stability in cross-temporal recognition scenarios [Luo et al. 2025]. Collectively, these methodologies are identified as essential enablers of Industry 4.0, playing a transformative role in contemporary engineering research and data management [Serey et al. 2023].

However, existing research is mostly limited to the prediction and alarm of the macro-health status of infrastructure or services; and the deep coupling of prediction capabilities with the control flow of specific business transactions (such as Saga) to achieve micro-level, real-time and proactive fault-tolerant decision-making is still Research gaps that have not been fully explored [Chen et al. 2016, Dean and Ghemawat 2008, Beyer and Laney 2012].

2.3 LSTM Networks in Time-Series Forecasting

System monitoring indicators (such as CPU utilization and memory consumption) are typical examples of time series data, and there is a strong correlation between the current state and the historical state. Recurrent Neural Networks (RNNs) are specially designed to process such sequence data, and the core goal is to capture the timing dependencies in the data. However, the traditional RNN architecture faces a key limitation when dealing with long sequence scenarios, it is difficult to effectively learn the long-term dependencies in the data due to the problem of gradient disappearance or explosion [Schmidhuber 2015]. To break through the long-term dependency modeling bottleneck of traditional RNN, Hochreiter and Schmidhuber proposed the long-term and short-term memory (LSTM) network in 1997, a special RNN variant specially designed to solve RNN timing memory defects [Hochreiter and Schmidhuber 1997]. The core innovative value of LSTM lies in the construction of independent cell state and fine gate control architecture (covering forgetting gate, input gate, output gate) [Gers et al. 2000]: this mechanism enables the network to selectively retain, update or discard historical information in each time step, so as to refine Extremely long-range dependency mode in quasi-capture time series. With its excellent time series modeling ability, LSTM has been widely used in financial market forecasting, weather forecasting, system anomaly detection and other fields [Malhotra et al. 2015, Hundman et al. 2018]. More importantly, empirical research on system monitoring scenarios shows that when LSTM processes CPU, memory and other indicators, it can not only effectively monitor the normal fluctuations and fault warning abnormalities, but also has strong robustness to data noise. It is these experimentally verified performance advantages that make it an ideal technical base for this study to build a failure precursor prediction model [Karim et al. 2017].

3 Proposed Framework and Methodology

3.1 PSFA Overall Architecture

The core design paradigm of the PSFA framework is to integrate predictive capabilities into the Saga orchestrator via a non-invasive mechanism. This design does not require

any changes to the business logic of existing microservices, and ensures the native nature of microservices at the architectural level. The overall workflow is shown in Figure 3. The architecture strategically embeds the call mechanism of PSFA prediction API in the traditional Saga arrangement process, and realizes active fault tolerance through ”prediction preposition”. Specifically, before performing each key transaction step, such as high-overhead or non-idempotent operations(such as deducting inventory and updating account balances), the Saga orchestrator will perform the following series of operations:

1. Data Collection: The orchestrator pulls a sequence of key performance indicators of the target service from the monitoring system (such as Prometheus) in real time (covering core indicators such as CPU utilization, memory occupancy, disk I/O rate, etc.). The sequence completely covers the sampling points of the last N time windows, for example, it contains 12 continuous observation data with an interval of 5 seconds, forming a chronological portrait of the service operation status.

2. Prediction Service Invocation: The orchestrator encapsulates this multi-dimensional time series data as input parameters and submits it to the deployed and running PSFA prediction service API interface.

3. Decision Making: The prediction API will output a quantified fault probability value, and its value range is strictly limited to [0.0, 1.0] closed interval. On this basis, the Saga orchestrator quantitatively compares this probability value with the risk threshold preset by the system (for example, set to 0.8) to assess the potential failure risk level of the current transaction execution path.

4. Flow Control: Based on the comparison results, the Saga orchestrator will trigger the following decision-making branches:

   (a) When the probability value is lower than the preset risk threshold, the service status is judged to be in a stable operating state. At this time, the Saga orchestrator will strictly follow the original transaction execution plan and continue to promote the current Saga transaction steps.

   (b) When the probability value breaks through the preset threshold, the service status is judged to be at a very high failure risk level. For this high-risk scenario, the Saga orchestrator will implement an active suspension strategy, not only to immediately terminate the global execution of the current Saga transaction, but also to synchronously trigger the compensation process for the completed transaction steps to avoid the ineffective resource investment of services on the verge of failure at the mechanism level. Prevent the waste of additional resources and the deterioration of the system status due to the call of the service that is about to collapse.

The architectural design that decouples the prediction model into independent services fundamentally guarantees the high scalability and maintainability of the whole framework.

3.2 High-Fidelity Data Generation

The construction of any reliable prediction model relies heavily on high-quality training data with precise labels. In the field of AIOps, obtaining large-scale and real-labeled

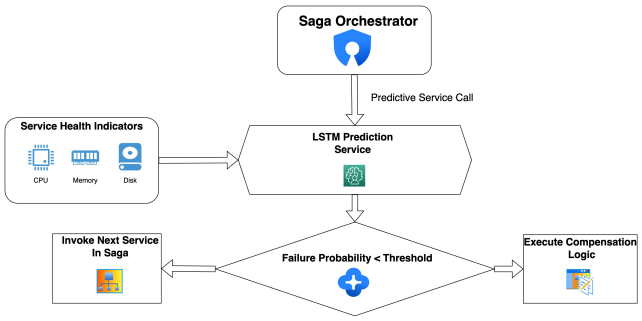


Figure 3: Workflow diagram of the PSFA framework. Before triggering the next service call, the Saga orchestrator will rely on the failure probability predicted by the real-time performance telemetry data of the LSTM service output based on the target service, the telemetry data covers core system-level indicators such as CPU utilization, memory occupancy rate, disk I/O rate, etc., through For the quantitative analysis of these indicators, the orchestrator can determine whether to continue the service call or start the compensation transaction to terminate the current transaction process in advance.

fault data sets has always been the core challenge that restricts the implementation of technology. In response to this challenge, this study designs and implements a high-fidelity data generation scheme, its core mechanism draws on the principle of controlled chaos engineering, actively injects controllable failure precursors through programming, and generates training samples required for supervised learning in a targeted manner. The specific generation logic is as follows:

1. Alternating Cycles: The generation script alternately generates two types of state cycles of random duration, ”normal operation cycle” and ”fault precursor cycle”. Its design goal is to reproduce the dynamic state switching process between the stable operation state and the near-fault state during real system operation.

2. Benign Peaks: During the normal operation cycle, the script actively injects short-term self-healing resource peaks with low probability, such as the temporary rise of CPU/memory caused by simulated service cold start or instantaneous traffic shock. Although such data points show sudden peaks on numerical characteristics, they are still accurately marked as ”normal (0)”. The core goal of this design is to strengthen the model’s ability to distinguish between transient and steady-state, so that it can effectively distinguish between momentary, benign resource fluctuations and malignant trends that foreshadow system risk, thereby significantly reducing the model’s false alarm rate.

3. Malignant Trends: Within the range of the failure precursor cycle, the three classic failure modes, whose failure process shows the typical characteristics of continuous deterioration, are simulated and reproduced by programmatic methods. All data points covered in this stage are marked by the system as ”Fault Premonition (1)”:

   (a) Memory Leak: Memory usage shows a slow and continuous linear growth trend. When the memory utilization exceeds the preset high water level threshold (such as 85%), it will be accompanied by severe CPU jitter, this setting aims

to simulate the typical scenario of rapid deterioration of system performance caused by frequent garbage collection (GC).

(b) CPU Saturation: The CPU utilization rate continues to climb and reaches saturation when approaching the threshold of 100%; at the same time, its fluctuation range (with standard deviation as a quantitative indicator) shows a significant upward trend. This simulation design is accurately reproduced, due to the overload of computing-intensive tasks, the system gradually reaches the typical operating state of its own processing capacity limit.

(c) I/O Bottleneck: Disk I/O busty-related indicators (such as iowait) show a sharp upward trend and continue to maintain a high level, during which it may be accompanied by a slight increase in CPU utilization. This simulation design aims to accurately reproduce the system response delay state caused by the log storm phenomenon or high-concurrency database write operation.

In addition, one of the core design points of this scheme is the systematic simulation of the dependence between indicators. For example, the later stage of the memory leakage process may cause significant abnormalities in the CPU index. Encoding these cross-index correlation patterns extracted from real failure scenarios into training data is of key significance to improving the authenticity and prediction accuracy of the model. To more intuitively present the intrinsic relationship of these data patterns, Figure 4 visualizes four typical generation scenarios.

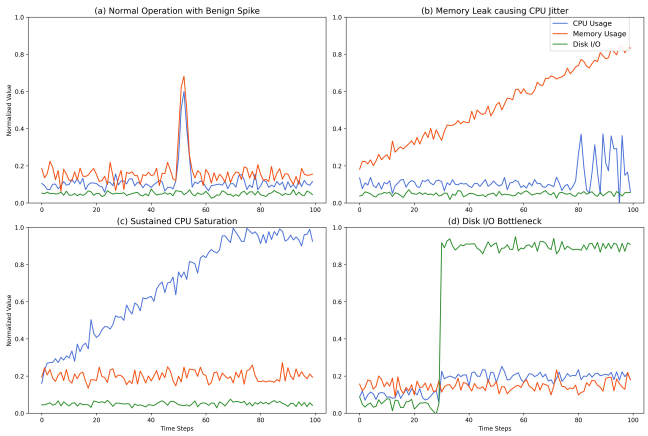


Figure 4: Visualization of simulated high-fidelity fault data. This figure shows four typical simulated time-series data patterns: (a) a normal operation cycle with a benign spike; (b) a memory leak trend and the CPU jitter it induces; (c) sustained CPU saturation; and (d) a disk I/O bottleneck. These patterns are designed to train the model to distinguish between normal fluctuations and true fault precursors and to learn the chain reactions between metrics.

3.3 Data Preprocessing and Leakage Prevention

In machine learning tasks involving time-series data, data leakage is a common and critical pitfall that must be avoided. It can cause a model to show artificially inflated performance during the evaluation phase [Kutz 2024], only to perform poorly in actual deployment. This leakage typically occurs during the preprocessing stage. For example, normalizing the entire dataset before splitting it unintentionally ”leaks” information about the test set’s data distribution to the training process. To completely prevent this issue, this study designed and strictly followed the industry best practice of ”split first, then normalize” [Brownlee 2018]. This workflow has been widely verified in the field of academic research and has been proven to be an effective way to prevent data leakage in time series analysis [Bergmeir et al. 2018]. Its specific implementation steps are as follows:

1. Chunking and Splitting: Firstly, the preprocessing script will carry out iterative traversal of the entire data set and execute precise annotations on the continuous time series data blocks corresponding to the ”normal” and ”fault” states. Then, these data blocks, not single data points, are used as randomly divided atomic units, divided into 80% training sets and 20% test sets. The core goal of this block-based division strategy is to ensure the integrity of the end-to-end evolutionary trajectory of any single failure event, thus avoiding the leakage of distributed information of the same event to the training set and the test set at the same time. This kind of design is crucial to maintain the sample independence of the test set, as well as the objectivity and generalized validity of the guarantee model evaluation.

2. Normalization: Then, the Min-Max scaling algorithm is used to perform a linear mapping operation on all eigenvalues and map them to the [0, 1] value range. The data normalization process is executed according to the Min-Max scaling method defined in **Equation (1)**:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

   To strictly follow the principle of leakage prevention, the scaling parameters (i.e. the minimum value and maximum value of each feature) are only learned from the training center (that is, obtained by fitting training data). Subsequently, the fitted scaler will be applied (that is, the conversion operation will be performed) to the training set and the test set. This process ensures that the test set always maintains the state of ”not being touched by the training set” throughout the normalization process, so as to completely avoid the risk of information leakage in the preprocessing stage from the methodology level.

3. Windowing: Finally, perform sliding window operations on the normalized training set and test set. The core goal of this step is to reconstruct the one-dimensional continuous time series data into a sample format that meets the needs of LSTM model supervision learning. In this study, the sample format is defined as [N, 12, 3], where N represents the total number of samples, 12 corresponds to the number of time steps, and 3 corresponds to the characteristic dimension. This setting shows that the model will predict the state of the system at the next point in time through the data sequence of the past 12 consecutive points, covering the three characteristics of CPU, memory and disk I/O.

3.4 LSTM Model Construction

Based on the PyTorch deep learning framework, this study builds an LSTM model for the second classification task of fault precursor detection. The design scheme of its specific network architecture is as follows:

1. Input Layer: Accept a three-dimensional input tensor with a shape of [N, 12, 3], where N is the batch size, 12 corresponds to the number of time steps, and 3 represents the number of features, CPU, memory and disk I/O respectively.
2. LSTM Layers: Consists of two stacked LSTM layers (num_layers=2), with the dimension of the hidden units set to 128 (hidden_size=128). A stacked architecture is used to enable the model to learn more complex and hierarchical abstract features from the time series.
3. Output Layer: The 128-dimensional hidden state vector from the last time step of the LSTM is fed into a fully connected layer (Linear Layer). This layer maps the high-dimensional features to a single scalar output.
4. Activation Function: A Sigmoid activation function is applied after the fully connected layer to compress the output value into the [0, 1] range, allowing it to be directly used as the predicted failure probability.

To provide a deeper explanation of the core working mechanism of LSTM, the key mathematical formulas for a single time step t are listed below [Graves 2012]. Given the input at the current time step X_t, the hidden state from the previous time step h_{t-1}, and the cell state C_{t-1}, the state transition and memory management of an LSTM unit are governed by the logic detailed in **Equations (2) through (7)**:

1. Forget Gate (f_t): Controls how much information from the previous cell state C_{t-1} is retained or forgotten.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{2}$$

2. Input Gate (i_t): Determines how much new information from the current input can be stored in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{3}$$

3. Candidate Cell State ($\tilde{C}_t$): Generates a candidate vector containing new information based on the current input and the previous hidden state.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{4}$$

4. Cell State Update (C_t): Updates the cell state by combining the output of the forget gate and the input gate.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{5}$$

5. Output Gate (o_t): Determines how much information from the updated cell state C_t can be used as the output for the current time step.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{6}$$

6. Hidden State Update (h_t): Computes the hidden state for the current time step by combining the output of the output gate and the updated cell state h_t.

$$h_t = o_t \odot \tanh(C_t) \tag{7}$$

Where W_f, W_i, W_C, and W_o are the weight matrices for the respective gates; b_f, b_i, b_C and b_o are the corresponding bias vectors; σ represents the Sigmoid activation function; tanh is the hyperbolic tangent function; $\odot$ denotes the Hadamard Product (i.e., element-wise multiplication).

4 Experiments and Results

4.1 Experimental Environment

1. Environment Configuration: All experiments in this study, including data generation, preprocessing, and model training, were conducted in a unified computational environment. The experimental workflow used Python 3.9 as the primary programming language and relied on its scientific computing ecosystem. Key libraries included Pandas and Numpy for data processing, Scikit-learn for data splitting and normalization, and Joblib for model serialization. To ensure the reproducibility of experimental results, this study sets fixed seeds (seed=42) for all random processes.

2. Model and Training Parameters: The model relies on the PyTorch 1.12 deep learning framework [Abadi et al. 2016] to complete the construction, training and evaluation, and encapsulates it as an API service with the help of the Flask 2.2 Web framework. The specific details of the training hyperparameters and the model configuration information are shown in Table 1. The ideal model configuration, including hidden units and layer count, was established through empirical evaluation and a preliminary grid search. Multiple hidden dimensions (64, 128, 256) and layer configurations (1, 2, 3) were assessed, with the selected parameters (hidden_size=128, layers=2) achieving the optimal equilibrium between training convergence and prediction accuracy.

Optimizer	Adam[Kingma and Ba 2015]
Loss Function	BCELoss
Learning Rate	0.001
Epochs	50
Batch Size	64
Dropout Rate	0.2

Table 1: Model Training Hyperparameters and Configuration.

4.2 Model Performance Evaluation

The proposed LSTM model was trained and evaluated using a dataset generated through the methodology described in Section 3. The raw dataset contains 15,000 discrete time points. After performing the block-based split (80% for training and 20% for testing) and applying a sliding window of size 12, the final processed dataset consists of 11,989 training samples and 2,989 testing samples. Each sample is a 3-feature temporal sequence (12×3), providing a robust foundation for capturing the evolutionary trajectories of system failures.

To comprehensively and systematically quantify the predictive performance of the model, this study selects the standard evaluation indicators common in the field of classification tasks, Precision, Recall and Overall Accuracy. These indicators are calculated based on the four basic quantities of the confusion matrix:

- True Positive (TP): The number of ”fault precursor” samples correctly classified as ”fault precursors”.
- True Negative (TN): The number of ”normal” samples correctly classified as ”normal”.
- False Positive (FP): The number of ”normal” samples incorrectly classified as ”fault precursors” (also known as a Type I error or a false alarm).
- False Negative (FN): The number of ”fault precursor” samples incorrectly classified as ”normal” (also known as a Type II error or a miss).

The mathematical definitions for these performance metrics are detailed in **Equations (8) through (11)**:

1. Precision: The proportion of samples that are truly ”fault precursors” among all samples predicted as ”fault precursors”, defined in **Equation (8)**.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{8}$$

2. Recall: The proportion of true ”fault precursor” samples that are successfully predicted, as shown in **Equation (9)**.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{9}$$

3. Accuracy: The proportion of correctly classified samples among all samples, which is computed by **Equation (10)**.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{10}$$

4. F1-Score: The harmonic mean of Precision and Recall, providing a balanced measure of the model's performance, as defined in **Equation (11)**:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{11}$$

The final performance evaluation results of the model on the independent test set are detailed in Table 2.

Precision	Recall	F1-Score	Accuracy
1	0.99	1	99.85%

Table 2: Performance Evaluation of the Model on the Test Set.

Results Analysis: As shown in Table 2, the model demonstrated excellent predictive performance on the independent test set. The performance on key metrics is as follows:

- Precision of 1.00: As calculated via **Equation (8)**, this means that all samples the model predicted as ”fault precursors” were actual fault precursors, achieving a zero false positive rate (False Positive Rate = 0). This is of core and key significance for the practical application of the PSFA framework, because it avoids the risk of the system terminating a healthy transaction due to a false alarm, thereby eliminating unnecessary business stoppages and resource consumption by design.
- Recall of 0.99: According to the definition in **Equation (9)**, this shows that the model accurately identifies 99% of the real failure precursors in the test set, and achieves an extremely low missed alarm rate. This is also of the core importance of the active risk avoidance system, it ensures that the vast majority of potential system failures can be captured and avoided in advance. At the same time, this recall rate, which is close to perfection but does not reach 1.00, also indirectly reflects the inherent challenge of the test data, further supporting the authenticity and credibility of the above evaluation results.

4.3 Analysis of the Model’s Ability to Identify Specific Fault Patterns

1. Experimental Design: To deeply analyze the generalization performance of the unified model (general_model.pth) across different fault types, this study designs supplementary experiments. The training stage of the model adopts a hybrid data set covering three types of ”malignant trends”, memory leakage, CPU saturation and I/O bottleneck. This experiment focuses on evaluating the performance of the model for the independent test set of a single failure mode. Based on this goal, the study has built three special test data sets, each data set only contains a specific failure mode and its corresponding normal operation cycle. Subsequently, the pre-trained general_model.pth model was used to evaluate the above three test sets one by one. The specific results are summarized in Table 3, and the visual analysis results are shown in Figure 5.
2. Results Analysis: The performance comparison results shown in Figure 5 reveal several key conclusions:
   (a) High Recall, Low Precision Trend: When dealing with independent I/O bottlenecks and memory leakage scenarios, the model shows almost extreme recall performance (0.98 and 0.87 respectively). This shows that the model accurately captures the core ”continuous deterioration” essential characteristics of these two failure modes and is highly sensitive to risk signals, there is a tendency to

Fault Pattern	Precision	Recall	F1-Score	Accuracy
CPU Saturation	0.09	0.28	0.13	25.92%
Memory Leak	0.23	0.87	0.36	41.13%
Disk I/O Bottleneck	0.27	0.98	0.42	61.43%
Mixed Pattern (Original Model)	1	0.99	1	99.85%

Table 3: Performance of the Unified Model on Single Fault Pattern Test Sets.

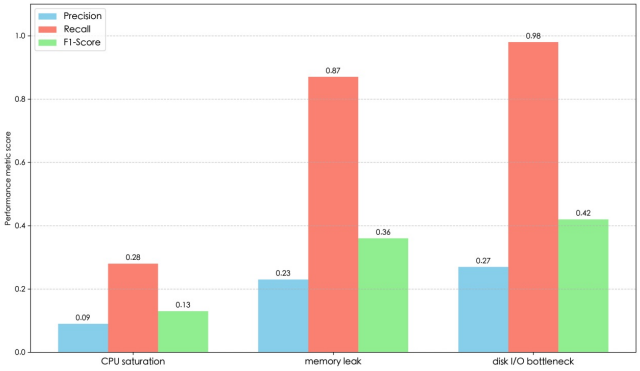


Figure 5: Comparative analysis of the performance indicators of the unified model in different single failure mode scenarios.

excessively mark suspected risk patterns as failures. However, the accuracy rate of the two is relatively low (0.27 and 0.23), resulting in a high false positive rate. This result may be attributed to the data distribution offset between the single failure test set and the hybrid training set, which ultimately leads to the blurring of the model decision boundaries.

(b) Limited Recognition of Independent CPU Saturation: The model showed the worst performance in dealing with pure CPU saturation scenarios, and its accuracy rate and recall rate fell to extremely low levels (0.09 and 0.28). A plausible explanation is that in the high-fidelity training data, the later stages of a memory leak are invariably accompanied by severe CPU jitter. This may have caused the model to establish an overly strong correlation between ”CPU metric anomalies” and ”concurrent symptoms of a memory issue”. Therefore, when faced with an independent CPU saturation problem that lacks preceding signs of memory pressure, the model’s generalization ability was limited.

(c) Necessity and Superiority of Mixed-Data Training: The core conclusion is that although the unified model’s performance was inconsistent in single-fault scenarios, it performed nearly perfectly (1.00 precision, 0.99 recall) on the original test set containing a mix of all fault patterns. This stark contrast strongly demonstrates that data containing only single, pure patterns is insufficient for training a general predictive model that is robust enough for complex environments. It is by learning the commonalities and differences among multiple fault patterns that the model was able to achieve excellent performance in a more realistic,

mixed scenario.

4.4 Sensitivity Analysis of Key Hyperparameter: Time Window Length

The length of the time window (time_step) is one of the core influencing parameters of the performance of the model in this study, which directly determines the scope of historical information that can be referenced when the model is predicted. If the window is too short, it may be difficult to capture the complete evolutionary trajectory of a failure due to insufficient context; if the window is too long, it may introduce irrelevant historical noise, which will increase the calculation cost. To verify the reasonableness of choosing 12 as the baseline window length in this study, and at the same time systematically examine the performance sensitivity of the model to this parameter, we carried out a series of comparative experiments.

1. Experimental Design: In this experiment, all other parameters (such as hidden layer dimension, learning rate, etc.) are fixed and unchanged. Only the core parameter of the length of the time window is adjusted, and the values are 4, 8, 12, 16, 24 and 36 in order. For each window length setting, the whole process of data preprocessing, model training and evaluation is fully carried out. The performance evaluation takes the F1 score (F1-Score) and Overall Accuracy (Overall Accuracy) of the ”fault precursor” category as the core evaluation indicators, and the experimental results are summarized in Table 4.

Time Window Length	Precision	Recall	F1-Score	Accuracy
4	0.97	1	0.98	99.33%
8	1	1	1	99.93%
12 (Baseline)	1	1	1	99.91%
16	1	0.98	0.99	99.68%
24	1	0.99	0.99	99.72%
36	1	0.99	0.99	99.67%

Table 4: Effect of Different Time Window Lengths on Model Performance.

2. Results Analysis: As shown in Figure 6, when the length of the time window is adjusted, the model performance shows significant regularity characteristics:
   (a) Optimal Performance Range: The experimental results of Table 4 show that when the length of the time window is set to 8 and 12, the model performance shows a significant peak. Under this configuration, the model’s F1 score for failure precursors reaches the theoretical optimal value of 1.00, and the overall accuracy rate is also at the optimal level. This result confirms that the sequence length of 8 to 12 time steps provides the LSTM network with the optimal amount of information needed to capture the key dynamic features of the transition from the normal state to the abnormal state.
   (b) Performance Decrease with a Short Window: When the window length is reduced to 4, the model performance shows a significant attenuation, and the accuracy rate drops to 0.97, which means that the model begins to show false positive

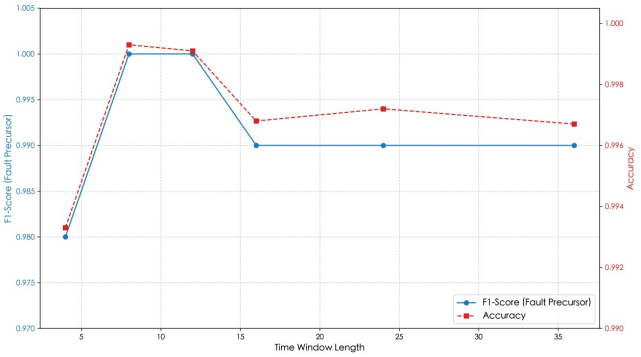


Figure 6: Evolutionary trend analysis of model performance adjustment with the length of the time window.

errors. This result is in line with the expected logic, the short sequence lacks sufficient contextual information, resulting in the model's inability to effectively distinguish the initial stage of the momentary "benign peak" and the continuous deterioration of the "malignant trend".

(c) Noise Interference from a Long Window: When the window length gradually increased from 16 to 36, the recall rate and F1 score of the model showed a mild but continuous downward trend. This result shows that longer historical data does not bring incremental value; on the contrary, it may mistakenly introduce outdated information unrelated to the current state as noise, slightly interfering with the accurate decision-making of the model.

In conclusion, this sensitivity analysis provides strong empirical support for the decision to study the length of the time window of 12 as the core experiment. The experimental results clearly reveal that there is an optimal window range (8-12 time steps); if the window is too short or too long, it will significantly weaken the accurate identification ability of the model, thus fully verifying the rationality and optimality of the parameter selection of this study.

4.5 API Interface Functional Verification

To validate the effectiveness of the deployed model in a simulated real-world invocation scenario, this section constructs two typical metric sequences, each with a length of 12 time steps, representing a "healthy state" and a "fault precursor state," respectively. The /predict interface, deployed on Flask, was then called using a curl command. The test results are shown in Table 5.

Results Analysis: The API interface's response was perfectly consistent with the model's expected behavior. For the stable, low-load sequence representing a healthy state, the model returned a failure probability close to 0; conversely, for the continuously deteriorating sequence simulating a fault precursor, the model returned a deterministic high-risk probability close to 1.0.

This result indicates that the deployed model is capable of accurately distinguishing between a healthy service state and a potential risk in a practical application. Therefore,

Scenario	Input Metric Sequence Description	Predicted Failure Probability
Healthy Sequence	CPU, memory, and I/O metrics fluctuate smoothly at low levels.	≈ 0.0
At-Risk Sequence	The memory metric shows a continuous linear increase, while the CPU metric begins to fluctuate sharply in the later stages.	≈ 1.0

Table 5: API Interface Verification Results.

this section functionally verifies the feasibility of using this predictive model as the core decision-making engine of the PSFA framework, providing the final functional support for the framework's practical implementation.

5 Discussion and Future Work

The results of this study clearly show that the application of deep learning-driven timing analysis technology to system indicator monitoring scenarios can achieve effective prediction of potential microservice failures, and then inject active fault tolerance into distributed transaction models such as Saga. The fundamental transformation of this paradigm dimension promotes the failure handling decision-making point from ”post-remediation” to ”pre-prevention”, which not only directly addresses the core significant challenges of traditional distributed transactions but also reduces the unnecessary loss of system resources, optimizes the user interaction experience, and enhances the overall stability of the service, which has significant practical guidance value. Unlike the traditional Saga optimization scheme (which mostly focuses on solving transaction isolation problems), the PSFA framework breaks out of the limitation of ”optimizing existing processes” and opens up a new path based on ”fault prevention” to ensure the ultimate consistency of distributed transactions by systematically improving system resilience. Nevertheless, there are still some limitations in this study, such as the insufficient coverage of simulated data for real scenes, and the model's ability to characterize complex fault chains needs to be strengthened, and these limitations precisely indicate the important direction of ”migrating to real scenes” and ”improving the generalization ability of models” for future research.

5.1 Limitations and Future Prospects of Data Sources

Firstly, the experimental data of this study are all generated based on simulation methods. Although the scheme has designed mechanisms such as benign peak injection and indicator chain response simulation, and tries its best to reproduce the fault evolution logic of the real scene, the fault mode in the real business environment is bound to present higher complexity and diversity, this is the inherent scene coverage limitation of the simulation method. Future research needs to focus on the collection and annotation of production environment data, further verify the generalization capability boundaries of the current model with the help of empirical data of real scenarios, and promote the iterative optimization of the model in the direction of ”real scene adaptability”.

5.2 Limitations and Future Prospects of Monitoring Dimensions

Second, the coverage of system indicators in this study is only limited to three basic system-level indicators (CPU, memory, disk I/O). However, in the real microservice environment, the root causes of failures are often scattered in a wider dimension, the existing index system is difficult to cover the potential triggers of complex failures. Future work can explore and build a multimodal prediction model [Petrova 2025]. For example, integrate application layer indicators (such as network latency, middleware queue length) to capture anomalies at the business logic level, and even mine textual anomalies from service logs in real time using natural language processing (NLP) technology. This expansion will realize a more comprehensive and multi-dimensional understanding of the health status of the service, and effectively make up for the characteristics of complex failure scenarios in the existing model.

5.3 Limitations and Future Prospects of Decision Strategies

In addition, the current framework has a single limitation of mechanism for the response strategy of high-risk prediction, only the fixed operation of ”immediate abortion of transactions” is adopted, and the risk classification response logic is not formed. Future research can explore intelligent decision-making mechanisms with scenario adaptability in response to this shortcoming. Specifically, dynamic response strategies can be designed based on the quantitative classification of predicted risks: low-risk scenarios maintain the normal execution of transactions to ensure business continuity; medium-risk scenarios try to switch to backup services or trigger service downgrades to balance fault avoidance and performance loss; only execute transactions in high-certainty and high-risk scenarios Stop, avoid excessive intervention. This kind of adaptive mechanism is expected to be further optimized through sequential decision-making optimization methods such as reinforcement learning to achieve a balance between ”risk response accuracy” and ”transaction execution efficiency”.

5.4 Computational Complexity Analysis

The runtime efficiency of the PSFA framework is a critical factor for its practical deployment. From a theoretical perspective, the computational complexity for a single inference of our LSTM model is $O(L \times n \times (d \cdot h + h^2))$, where $L = 12$ represents the time window length, $d = 3$ is the input feature dimension (CPU, memory, and disk I/O), $h = 128$ is the hidden state dimension, and $n = 2$ denotes the number of stacked LSTM layers.

Given these parameters, the model performs lightweight matrix-vector multiplications that can be completed in less than 10 milliseconds on a standard commodity CPU. This theoretical efficiency ensures that the ”predictive preposition” mechanism does not become a bottleneck for the Saga orchestrator. The following subsection further discusses the systemic performance overhead and corresponding deployment optimizations.

5.5 Performance Overhead and Deployment Strategies

In addition, the PSFA framework integrates synchronous network calls into the key path of Saga transactions, this mechanism design inevitably leads to additional delays. For business scenarios with high throughput or highly sensitive to latency, the performance overhead caused by such network calls needs to be carefully evaluated and optimized. The future optimization direction can focus on the following core dimensions:

1. Strategic Prediction: Instead of performing predictions for every step, prediction logic could be applied only before critical steps that are high-cost, non-idempotent, or have a history of frequent failures, to balance the overhead and benefits.
2. Low-Latency Deployment: Deploying the prediction service in close physical proximity to the Saga orchestrator (e.g., as a Sidecar container) to minimize network communication latency as much as possible.
3. Asynchronous Prediction Mechanism Exploration: For non-core business operations, exploring an asynchronous invocation of the prediction service could be an option. However, this would introduce a ”time gap” problem between decision-making and execution, meaning the transaction step might have already been executed by the time the prediction result returns. How to handle this timing conflict is, in itself, a topic worthy of in-depth research.

Furthermore, future work will involve comprehensive comparative studies between the PSFA framework and other emerging proactive fault tolerance strategies to further validate its relative advantages in diverse operational scenarios.

5.6 Exploration of Model Architecture

Finally, although LSTM performed exceptionally well in this task, novel network architectures represented by the Transformer have recently shown significant potential in capturing long-sequence dependencies [Vaswani et al. 2017]. Future research can deeply explore the migration of more advanced model architectures such as Transformer (and its variants such as Informer) to the PSFA framework [Zhou et al. 2021], with the goal of promoting the further optimization of predictive performance.

6 Conclusion

In view of the inherent resource waste and high expense significant challenges of the passive compensation mechanism of the traditional Saga pattern, this study proposes and realizes the Proactive Saga Failure Avoidance (PSFA) framework. By embedding a predictive analysis module based on long-term and short-term memory network (LSTM) in the key decision-making nodes of Saga transactions, the framework realizes a fundamental change in the fault tolerance paradigm from ”compensation after failure” to ”active avoidance before failure”.

By designing a high-fidelity fault simulation scheme and a rigorous data leakage prevention pretreatment process, this study has successfully trained a high-performance LSTM prediction model. The key experimental results show from the quantitative index level that the model can accurately identify the failure precursors with an accuracy rate of 1.00 and a recall rate of 0.99, and then clearly distinguish between benign system fluctuations and malignant degradation trends, this result verifies the practicality of the scheme from the dimension of model capability. The final API function verification experiment further confirmed the effectiveness of the framework in simulating real-world call scenarios and provided empirical support for the implementation of its project.

In summary, in response to the core issue of ”deep integration of predictive analysis into distributed transaction management”, this study puts forward a technical path that is both feasible and targeted, and builds a solid empirical foundation based on system

experiments. This result not only clearly verifies the effectiveness of predictive analysis in distributed transaction scenarios, but also highlights its great potential for building the next generation of ”intelligent and flexible” distributed systems from the level of technical paradigms, and provides a technical blueprint for engineering and industrial applications in this field.

7 Data Availability

The dataset and the simulation scripts used in this study are available from the corresponding author upon reasonable request.

References

[Abadi et al. 2016] Abadi, M., Barham, P., Chen, J., et al.: “TensorFlow: a system for large-scale machine learning”; Proc. 12th USENIX Conference on Operating Systems Design and Implementation (2016).

[Bergmeir et al. 2018] Bergmeir, C., Hyndman, R. J., Koo, B.: “A note on the validity of cross-validation for evaluating autoregressive time series prediction”; Computational Statistics & Data Analysis, 120 (2018), 70-83.

[Beyer and Laney 2012] Beyer, M. A., Laney, D.: “The Importance of ’Big Data’: A Definition”; Gartner, 2012.

[Breiman 2001] Breiman, L.: “Random Forests”; Machine Learning, 45, 1 (2001), 5-32.

[Brownlee 2018] Brownlee, J.: “Deep Learning for Time Series Forecasting”; Machine Learning Mastery, 2018.

[Chen et al. 2016] Chen, J., Li, K., Tang, Z., Bilal, K., Yu, S.: “A Parallel Random Forest Algorithm for Big Data in a Spark Cloud Computing Environment”; IEEE Transactions on Parallel and Distributed Systems, 28, 4 (2016), 919-933.

[Correia and Jardim 2025] Correia, D., Jardim, S.: “A deep learning approach for face detection”; Procedia Computer Science, 263 (2025), 34-41.

[Cortes and Vapnik 1995] Cortes, C., Vapnik, V.: “Support-vector networks”; Machine Learning, 20, 3 (1995), 273-297.

[Dean and Ghemawat 2008] Dean, J., Ghemawat, S.: “MapReduce: simplified data processing on large clusters”; Communications of the ACM, 51, 1 (2008), 107-113.

[Garcia-Molina and Salem 1987] Garcia-Molina, H., Salem, K.: “Sagas”; ACM SIGMOD Record, 16, 3 (1987), 249-259.

[Gers et al. 2000] Gers, F. A., Schmidhuber, J., Cummins, F.: “Learning to Forget: Continual Prediction with LSTM”; Neural Computation, 12, 10 (2000), 2451-2471.

[Goodfellow et al. 2016] Goodfellow, I., Bengio, Y., Courville, A.: “Deep Learning”; MIT Press, 2016.

[Graves 2012] Graves, A.: “Supervised Sequence Labelling with Recurrent Neural Networks”; Springer, 2012.

[Hangaragi and Singh 2023] Hangaragi, S., Singh, T.: “Face detection and Recognition using Face Mesh and deep neural network”; Procedia Computer Science, 218 (2023), 741-749.

[Hochreiter and Schmidhuber 1997] Hochreiter, S., Schmidhuber, J.: “Long Short-Term Memory”; Neural Computation, 9, 8 (1997), 1735-1780.

[Hundman et al. 2018] Hundman, G., Constantinou, V., Laporte, C., Colwell, I., Soderstrom, T.: "Detecting Spacecraft Anomalies Using LSTMs and Automatic Thresholding"; Proc. 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (2018).

[Irvanizam et al. 2023] Irvanizam, I., Horatius, I., Sofyan, H.: "Applying Artificial Neural Network Based On Backpropagation Method For Indonesian Sign Language Recognition"; International Journal of Computing and Digital Systems, 14, 1 (2023), 975-985.

[Karim et al. 2017] Karim, F., Majumdar, S., Darabi, H., Chen, S.: "LSTM Fully Convolutional Networks for Time Series Classification"; IEEE Access, 6 (2017), 1662-1669.

[Kingma and Ba 2015] Kingma, D. P., Ba, J.: "Adam: A Method for Stochastic Optimization"; Proc. 3rd International Conference on Learning Representations (ICLR) (2015).

[Kutz 2024] Kutz, J.: "What is Data Leakage in Machine Learning?"; IBM Technology, 2024.

[LeCun et al. 2015] LeCun, Y., Bengio, Y., Hinton, G.: "Deep learning"; Nature, 521, 7553 (2015), 436-444.

[Luo et al. 2025] Luo, T., Wang, C., Wang, D., et al.: "Cross Temporal Scale Pig Face Recognition Based on Deep Learning"; Journal of Agriculture and Food Research, 102002 (2025).

[Malhotra et al. 2015] Malhotra, P., Vig, L., Shroff, G., Agarwal, P.: "Long Short Term Memory Networks for Anomaly Detection in Time Series"; Proc. 23rd European Symposium on Artificial Neural Networks (2015).

[Nadareishvili et al. 2016] Nadareishvili, I., Mitra, R., McLarty, M., Amundsen, M.: "Microservice Architecture: Aligning Principles, Practices, and Culture"; O'Reilly Media, 2016.

[Newman 2015] Newman, S.: "Building Microservices: Designing Fine-Grained Systems"; O'Reilly Media, 2015.

[Petrova 2025] Petrova, E.: "Deep Learning-Powered Fault Prediction in Microservices Ecosystems Under Continuous DevOps Operations"; International Journal of Research in Computer Applications and Information Technology (IJRCAIT), 2025.

[Richardson 2018] Richardson, C.: "Microservices Patterns: With examples in Java"; Manning Publications, 2018.

[Salfner et al. 2010] Salfner, F., Lenk, M., Malek, M.: "A survey of online failure prediction methods"; ACM Computing Surveys (CSUR), 42, 3 (2010), 1-42.

[Schmidhuber 2015] Schmidhuber, J.: "Deep learning in neural networks: An overview"; Neural Networks, 61 (2015), 85-117.

[Serey et al. 2023] Serey, J., Alfaro, M., Fuertes, G., et al.: "Pattern recognition and deep learning technologies, enablers of industry 4.0, and their role in engineering research"; Symmetry, 15, 2 (2023), 535.

[Soldani et al. 2018] Soldani, J., Tamburri, D. A., Heuvel, W. J. van den: "The Pains and Gains of Microservices: A Systematic Grey Literature Review"; Journal of Systems and Software, 146 (2018), 215-232.

[Vaswani et al. 2017] Vaswani, A. et al.: "Attention is All You Need"; Advances in Neural Information Processing Systems, 30 (2017).

[Wu 2024] Wu, J.: "Challenges and Solutions of Distributed Transactions in Medical Software under Microservices Architecture"; Advances in Computer, Signals and Systems, 8, 5 (2024), 119-125.

[Zhang et al. 2018] Zhang, Q., Yang, L. T., Chen, Z., Li, P.: "A Survey on Deep Learning for Big Data"; Information Fusion, 42 (2018), 146-157.

[Zhou et al. 2021] Zhou, H. et al.: "Informer: Beyond Efficient Transformer for Long Sequence

Time-Series Forecasting"; Proc. AAAI Conference on Artificial Intelligence, 35, 12 (2021), 11106-11115.